

AI-Powered Product Development — Facilitator Cheat Sheet

Live DMS exercise · 6 SDLC phases · Industrial DevOps Now

Prime the chat once (before Phase 1): "We're running a live workshop. Keep each answer concise and skimmable — bullets over prose."

Fresh chat, in order. Rhythm: **paste** → **read aloud** → **ask the room** → **move on**. For **Develop** & **Test**, use a tool that can run code (Code Interpreter / Claude / Jupyter) — or fall back to **simulation**.

PHASE	PASTE INTO THE AI	ASK THE ROOM
1 · Plan	Vision, stakeholder needs, capability roadmap, top risks/constraints for the DMS.	<i>Does this capture the real goal? What's missing?</i>
2 · Analyze	Requirements spec: functional + non-functional (latency, accuracy, privacy), assumptions, acceptance criteria; flag gaps.	<i>What would you challenge? What did it surface?</i>
3 · Design	Architecture (sensors → AI → decision logic → HMI/vehicle), interfaces, failure modes, safety/privacy; then a SysML v2 model that traces to requirements.	<i>Where could this go wrong, and how does it stay safe?</i>
4 · Develop	Runnable Python: state + response enums, a DMS class with detect-assess-respond (confidence gating, escalation), event log. Show + run it.	<i>Real code — no separate dev-team handoff.</i>
5 · Test	pytest for the requirements; run green/red . Then simulate scenarios (attentive, drowsy, distracted, sunglasses) and print decisions.	<i>Which test must pass before shipping?</i>
6 · Deploy	OTA rollout + rollback, monitoring metrics (accuracy, false-alarm rate, trust), release notes, customer impact, success criteria.	<i>How many teams would this normally take?</i>

Discussion moments (no tools — show of hands / call-out)

- **Opening:** Where does your org spend the most engineering effort?
- **Setup:** What slows product delivery the most today? (jot on a board)
- **Midpoint (after Develop):** Which SDLC phase benefited most from AI so far?
- **Future-state:** Where would you pilot AI first?
- **Closing:** What is the biggest opportunity?

Closing discussion

- What surprised you? · What would you challenge before trusting it?
- Where is human expertise still required? · How many teams would this traditionally need?
- What handoffs did we eliminate? · It's a safety system — what changes that AI helped design *and run* it?

If running long: collapse Design (skip the SysML half) or shorten Test to just the pytest run. **Never skip Develop + Test running live** — the working code is the payoff.